

# 파머완 6장 차원축소

## 01. 차원 축소 개요

- 차원 ↑ 데이터 포인트 간 거리 ↑ ⇒ sparse한 구조
- 피쳐 多 피쳐 간 상관관계 높을 가능성 ↑ ( multicollinearity )
- 수십개 이상의 피쳐 ⇒ 3차원 이하로 차원 축소 ; 직관적 데이터 해석 가능

### # 종류

#### 1. 피쳐 선택( feature selection )

- 불필요한 피쳐 제거, 주요 피쳐만 선택

#### 2. 피쳐 추출( feature extraction )

- 기존 피쳐를 저차원의 중요 피쳐로 압축해 추출
- 단순 압축 ✗, 피쳐를 함축적으로 더 잘 설명할 수 있는 다른 공간으로 매핑  
= 잠재적 요소( Latent Factor ) 추출

### # 차원 축소의 목적

- 차원 축소를 통해 더 데이터를 잘 설명할 수 있는 잠재적인 요소 추출

### # 대표적인 차원 축소 알고리즘

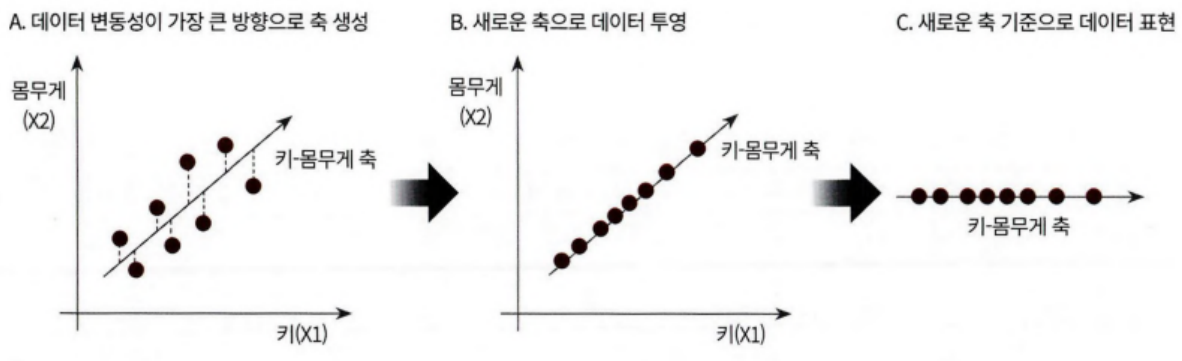
- PCA
- LDA
- SVD
- NMF

### ## 사용 영역

1. 매우 많은 픽셀로 이루어진 이미지 데이터에서 잠재된 특성 피쳐로 도출,  
 함축적 형태의 이미지 변환 & 압축 수행  
 압축된 이미지는 원본이미지보다 훨씬 적은 차원, overfitting ⬇️ 예측 성능 ⬆️
2. 텍스트 문서의 숨겨진 의미 추출  
 문서 내 단어 구성에 숨겨져 있는 Semantic 의미/ topic 찾아내기  
 SVD, NMF - Semantic Topic 모델

## 02. PCA ( Principal Component Analysis )

- 여러 변수 간 존재하는 상관관계를 이용해 이를 대표하는 주성분(principal component) 추출
- 가장 높은 분산을 가지는 데이터의 축을 찾아 이 축으로 차원 축소,  
 ⇒ PCA의 주성분



- 첫 번째 벡터 축 - 가장 큰 데이터 변동성(Variance) 기반 생성
- 두 번째 벡터 축 - 첫 번째 축의 법선 벡터
- 세 번째 벡터 축 - 두 번째 축의 법선 벡터

⇒ 이렇게 생성된 벡터 축에 원본 데이터 투영하면 벡터 축 개수 만큼의 차원으로 원본 데이터 차원 축소됨.

- `from sklearn.decomposition import PCA`

### # 선형대수 관점에서의 PCA 해석

$$C = P \Sigma P^T$$

$C$  : covariance matrix ( 공분산 행렬 )

$n \times n$  꼴(정방), 대칭행렬(

$P = P^T$ )

cf. 대칭행렬은 항상 고유벡터 → 직교행렬, 고유값 → 정방행렬 대각화 가능

$P$  :  $n \times n$ 의 직교 행렬 ( 고유벡터 직교 행렬 )

$\Sigma$  :  $n \times n$  정방 행렬(Square Matrix) ( 고유벡터 정방 행렬 )

$P^T$  :  $P$ 의 전치 행렬(transpose 행렬) ( 고유벡터 직교 행렬 )

$$C = [e_1 \cdots e_n] \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^t \\ \cdots \\ e_n^t \end{bmatrix}$$

$C = (\text{고유벡터 직교 행렬}) * (\text{고유값 정방 행렬}) * (\text{고유벡터 직교 행렬})$

$= (1,n) * (n,n) * (n,1) = (1,1)$  특정

$e_i$ 는  $i$ 번째 고유벡터,  $\lambda_i$ 는  $i$ 번째 고유벡터(eigen vector)의 크기

$e_1$ 은 가장 분산이 큰 방향을 가진 고유벡터,  $e_2$ 는  $e_1$ 에 수직, 그 다음으로 분산이 제일 큼

## # PCA Step

1. input dataset 공분산 행렬 생성
2. eigen vector, eigen value 계산
3. eigen value가 가장 큰 순으로  $k$ (PCA 변환 차원 수)만큼 eigen vector 추출
4. 3.에서 추출된 벡터로 새로운 input data 변환

## 03. LDA( Linear Discriminant Analysis )

- 개별 클래스를 분별할 수 있는 기준을 최대한 유지하면서 차원 축소

- 클래스간 분산(between-class scatter) & 클래스 내부 분산(within-class scatter )  
비율 maximize
- 클래스간 분산 ⬆, 클래스 내 분산 ⬇
- from sklearn.discriminant\_analysis import LinearDiscriminantAnalysis

## # LDA Step

1. 클래스 내/클래스 간 분산 행렬 구하기  
( 입력 데이터 결정값 클래스별로 개별 피처의 평균 벡터 기반 )
2.  $S_w$  = 클래스 내 분산 행렬

$S_B$  = 클래스 간 분산 행렬  
분해

$$S_W^T S_B = \begin{bmatrix} e_1 & \cdots & e_n \end{bmatrix} \begin{bmatrix} \lambda_1 & \cdots & 0 \\ \cdots & \cdots & \cdots \\ 0 & \cdots & \lambda_n \end{bmatrix} \begin{bmatrix} e_1^T \\ \cdots \\ e_n^T \end{bmatrix}$$

3. eigen value 큰 순으로 K( LDA 변환 차원수)개 eigen vector 추출
4. eigen vector로 새 input data 변환

## 04. SVD( Singular Value Decomposition )

- PCA - nxn 행렬만 분해 가능  
SVD - mxn 행렬도 분해 가능 ( 특이값 분해)
- numpy.linalg.svd (SVD 분해)

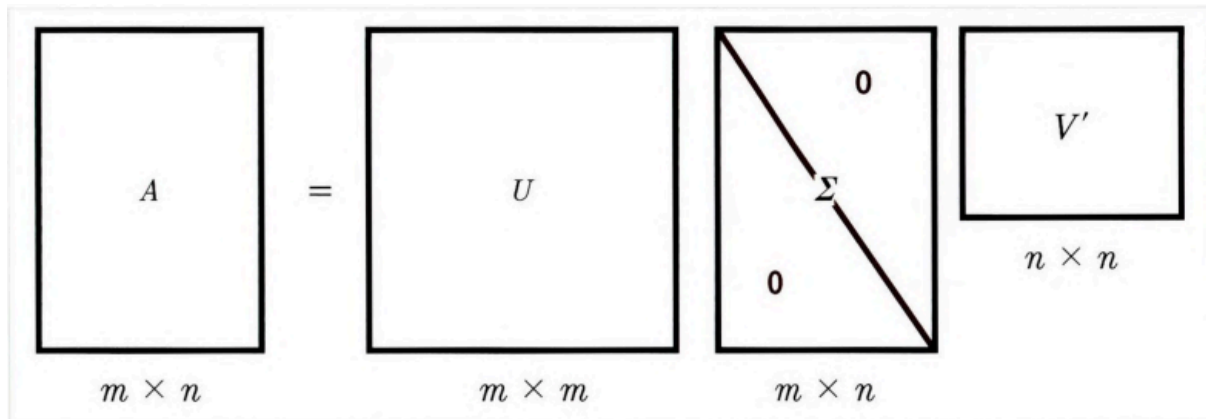
$$A = U \Sigma V^T$$

- $A$  : mxn 행렬

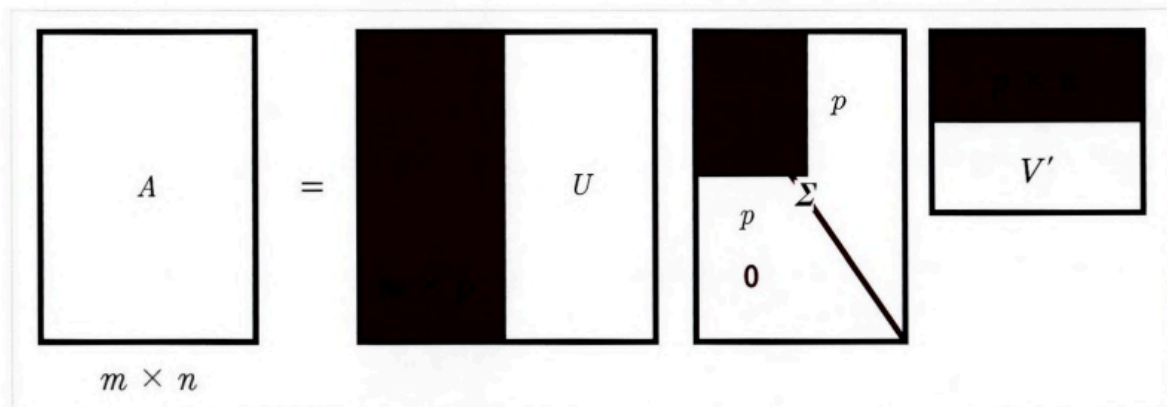
$\Sigma$  : 대각행렬 ( $m \times n$  행렬)

$U, V$  : 특이벡터 (singular vector) - 모든 특이 벡터는 직교함. ( $U$ 는  $m \times m$ 행렬)

$V^T$ :  $n \times n$ 행렬



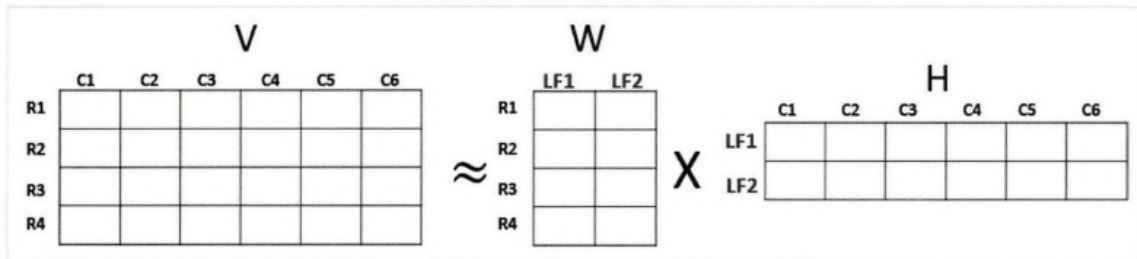
## # Truncated SVD



- $\Sigma$ 의 대각 원소 중 상위 몇개만 추출
- $A$ :  $m \times n$ ,  $\Sigma$ :  $p \times p$ ,  $U$ :  $m \times p$ ,  $V^T$ :  $p \times n$
- 원본 행렬 정확하게 다시 복구 불가능, but 꽤 잘 근사 가능
- `from scipy.sparse.linalg import svds` - 희소행렬, truncated SVD 지원  
from  
`scipy.linalg import svd` - 일반 SVD 지원
- `from sklearn.decomposition import TruncatedSVD`

## 05. NMP( Non-Negative Matrix Factorization )

- Truncated SVD같이 행렬 근사 방식 변형(Low-Rank Approximation)
- 원본 행렬 내 모든 원소 값  $> 0 \Rightarrow$  두 양수 행렬로 분해



- $H$  : 원본 행렬과 행 크기가 같은 행렬 ( 넓적 )  
잠재 요소가 원본 열(속성)으로 어떻게 구성됐는지 나타냄

$W$  : 원본 행렬과 열 크기가 같은 행렬 ( 길쭉 )

잠재 요소의 값이 원본 행에 대해 얼마나 되는지 나타냄

- `from sklearn.decomposition import NMF`
- 추천 영역에 활발히 적용 - 사용자-평가 분해해서 평가X 상품 잠재 요소 추출, 평가순위 예측, 높은 순위 예측 상품 추천 ( Latent Factoring 기반 추천 방식 )
- 이미지 압축을 통한 패턴 인식, 텍스트의 토픽 모델링 기법, 문서 유사도 및 클러스터링 ( SVD 유사 )